



Laboratório 2: Sockets TCP e UDP em Java

25/08/2026

i Nota

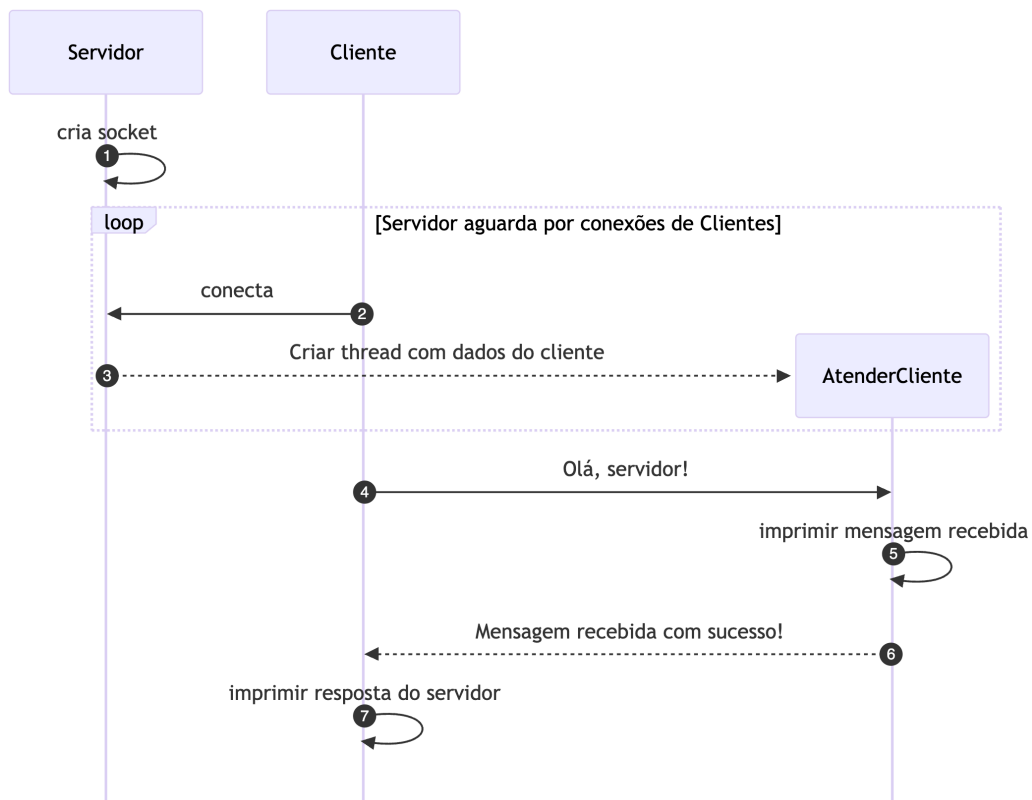
Java possui diferentes APIs para lidar com operações de I/O. A API Java NIO.2 permite desenvolver aplicações com comunicação assíncrona, porém seu entendimento demanda mais tempo. Veja mais detalhes em <https://docs.oracle.com/en/java/javase/25/core/java-nio.html>.

1 Sockets TCP

1.1 Olá Mundo

Neste exemplo o cliente, após conectar no servidor TCP, envia uma mensagem de saudação (String) ao servidor. O servidor recebe a mensagem, a imprime no console, envia uma resposta ao cliente (String) e finaliza a conexão. O cliente aguarda o servidor responder, imprime a resposta no console e finaliza a conexão. Na Figura 1 temos o diagrama de sequência do exemplo.

Figura 1: Diagrama de sequência do exemplo Olá Mundo



Listagem 1: Servidor TCP

```
package engtelecom.std.servidor;

import module java.base; // Java 25: importação de módulo

public class Servidor {
    private static final int PORTA = 12345;

    public static void main(String[] args) {
        System.out.println("\u2591\u2592\u2592 Iniciando o servidor \u2591\u2592\u2592");

        // ExecutorService com virtual threads para atender clientes de forma eficiente
        // Virtual threads permitem lidar com milhares de conexões simultâneas
        try (ServerSocket servidor = new ServerSocket(PORTA);
            var executor = Executors.newVirtualThreadPerTaskExecutor()) {

            System.out.printf("Aguardando conexões em %s:%d\n",
                servidor.getInetAddress().getHostAddress(), PORTA);
            System.out.println("Pressione CTRL+C para encerrar.\n");

            while (!Thread.currentThread().isInterrupted()) {
                // accept() bloqueia a thread principal
                Socket clienteSocket = servidor.accept();
                // Submete o atendimento do cliente a uma virtual thread
                executor.submit(new AtenderCliente(clienteSocket));
            }
        } catch (IOException e) {
            System.err.println("Erro no servidor: " + e.getMessage());
        }
    }
}
```

Listagem 2: Thread para atender cliente

```
package engtelecom.std.servidor;

import module java.base; // Java 25: importação de módulo

public record AtenderCliente(Socket cliente) implements Runnable {

    @Override
    public void run() {
        var endereco = cliente.getInetAddress().getHostAddress();
        var porta = cliente.getPort();
        System.out.printf("Cliente conectado: %s:%d\n", endereco, porta);

        try (cliente;
            var reader = new BufferedReader(
                new InputStreamReader(cliente.getInputStream(), StandardCharsets.UTF_8));
            var writer = new BufferedWriter(
                new OutputStreamWriter(cliente.getOutputStream(), StandardCharsets.UTF_8))) {

            String mensagem = reader.readLine();
            System.out.printf("[%s:%d] Mensagem recebida: %s\n", endereco, porta, mensagem);
            // Resposta ao cliente
            writer.write("Mensagem recebida com sucesso!");
            writer.newLine();
            writer.flush();
        } catch (IOException e) {
            String msg;
            if (e instanceof SocketException se && se.getMessage().contains("Connection reset")) {
```

```

        msg = "Cliente desconectou abruptamente\n";
    } else {
        msg = String.format("Erro: %s\n", e.getMessage());
    }
    System.err.print(String.format("[%s:%d] %s", endereco, porta, msg));
}
System.out.printf("Cliente encerrado: %s:%d\n", endereco, porta);
}
}

```

Listagem 3: Cliente TCP que envia uma String

```

package engtelecom.std.cliente;

import module java.base; // Java 25: importação de módulo

public class Cliente {

    record ConexaoConfig(String host, int porta) {

        // Design pattern "Factory Method"
        static ConexaoConfig of(String[] args) {
            return switch (args.length) {
                case 2 -> new ConexaoConfig(args[0], Integer.parseInt(args[1]));
                default -> new ConexaoConfig("localhost", 12345);
            };
        }
    }

    public static void main(String[] args) {
        System.out.println("\n\n\u2591\u2592\u2592 Iniciando o cliente \u2591\u2592\u2591\n");

        var config = ConexaoConfig.of(args);

        // try-with-resources para garantir o fechamento automático dos recursos
        try (var socket = new Socket(config.host(), config.porta());
            var reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
            var writer = new BufferedWriter(
                new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8))) {

            System.out.printf("Conectado ao servidor %s:%d\n", config.host(), config.porta());

            // Envio
            writer.write("Olá, servidor!");
            writer.newLine();
            writer.flush();

            // Recepção
            String resposta = reader.readLine();
            System.out.println("Resposta do servidor: " + resposta);

        } catch (IOException e) {
            System.err.println("Erro: " + e.getMessage());
        }
    }
}

```

Pergunta

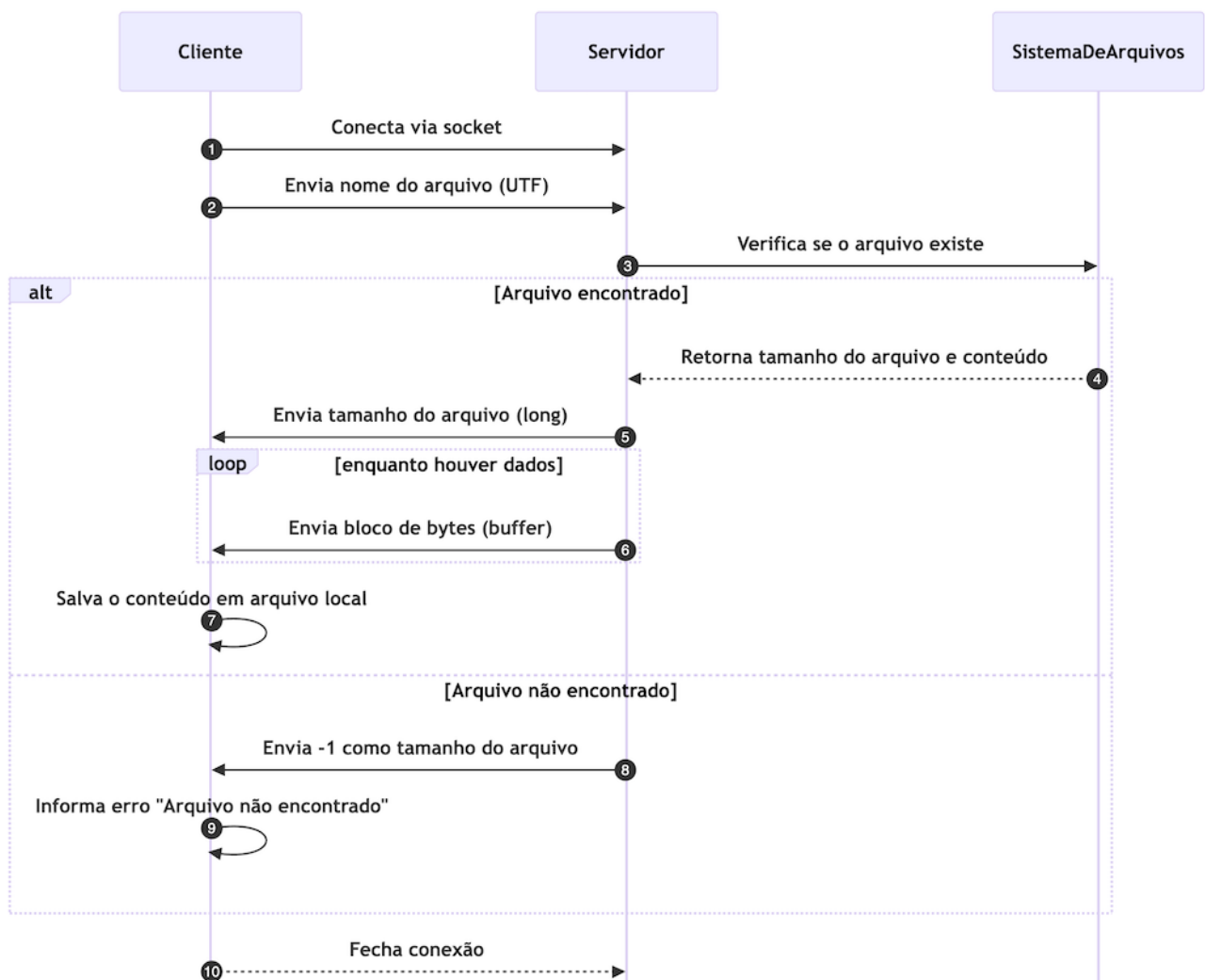
Seria possível usar o `netcat` para interagir com o Servidor ou com o cliente TCP desenvolvido neste laboratório? Caso seja, faça a demonstração e, se necessário, realize as modificações necessárias nos códigos. Caso não seja, explique o motivo.

1.2 Exercício: Transferência de arquivos de tamanho arbitrário

Com base no código da Subseção 1.1, faça um programa que permita transferir arquivos de tamanho arbitrário entre o cliente e o servidor. O cliente envia, ao servidor, o nome do arquivo que deseja receber. Se o arquivo existir, então o servidor envia o tamanho do arquivo e depois o arquivo (dividido em blocos de bytes de tamanho 4.096). Se não existir, então o servidor envia o valor -1 para o cliente. O cliente deve receber o tamanho do arquivo e, se o tamanho for maior que zero, deve receber o arquivo (em blocos de bytes de tamanho 4.096).

Na Figura 2 é apresentado o diagrama de sequência com as trocas de mensagens entre cliente e servidor. O servidor deverá ser *multithread*, ou seja, deve atender vários clientes simultaneamente.

Figura 2: Diagrama de sequência do exemplo transferência de arquivo



Trechos de códigos Java necessários para o desenvolvimento do exercício

Listagem 4: Trecho de código necessário para o servidor

```
// assumindo que o clientSocket já foi criado previamente
try (var clientSocket ;
    var dis = new DataInputStream(clientSocket.getInputStream());
    var dos = new DataOutputStream(clientSocket.getOutputStream())) {

    String nomeArquivo = dis.readUTF();
    System.out.println("Arquivo solicitado: " + nomeArquivo);

    if (Files.exists(Path.of(nomeArquivo))) {
        // Se o arquivo existe, envie seu tamanho e conteúdo
        long tamanho = Files.size(Path.of(nomeArquivo));
        dos.writeLong(tamanho);
        dos.flush();

        long bytesEnviados = Files.copy(Path.of(nomeArquivo), dos);
        dos.flush();
        System.out.printf("Enviado: %s (%d bytes)%n", nomeArquivo, bytesEnviados);
    } else {
        // Se o arquivo não existe, envie -1 para indicar erro
        dos.writeLong(-1);
        dos.flush();
        System.out.println("Arquivo não encontrado: " + nomeArquivo);
    }
}
```

Listagem 5: Trecho de código necessário para o cliente

```
String nomeArquivo = "slides.pdf";

try (var socket = new Socket(enderecoDoServidor, portaDoServidor);
    var dos = new DataOutputStream(socket.getOutputStream());
    var dis = new DataInputStream(socket.getInputStream())) {

    // Envia o nome do arquivo desejado
    dos.writeUTF(nomeArquivo);
    dos.flush();

    // Indica o caminho de destino para salvar o arquivo recebido
    Path destino = Path.of(nomeArquivo)

    // Lê a resposta do servidor (tamanho do arquivo ou -1 para indicação de erro)
    long tamanho = dis.readLong();

    if (tamanho > 0) {
        // Irá salvar o arquivo recebido no caminho especificado, substituindo se já existir
        long bytesRecebidos = Files.copy(dis, destino, StandardCopyOption.REPLACE_EXISTING);

        System.out.printf("Arquivo salvo: %s (%d / %d bytes)%n",
            nomeArquivo, bytesRecebidos, tamanho);
    } else {
        System.out.println("O servidor informou que o arquivo não existe: " + nomeArquivo);
    }
}
```

2 Sockets UDP

Listagem 6: Servidor UDP

```
package engtelecom.std.udp;

import module java.base; // Java 25: importação de módulo

public class ServidorUdp {

    private static final int    PORTA        = 9876;
    private static final int    BUFFER_SIZE  = 1024;
    private static final String RESPOSTA     = "Olá, eu sou o servidor UDP!";

    record PacoteRecebido(String mensagem, InetAddress origem, int portaOrigem) {}

    sealed interface Resultado permits Resultado.Sucesso, Resultado.Falha {
        record Sucesso(PacoteRecebido pacote) implements Resultado {}
        record Falha(String motivo)           implements Resultado {}
    }

    private static Resultado receberPacote(DatagramSocket socket, byte[] buffer) {
        var pacote = new DatagramPacket(buffer, buffer.length);
        try {
            socket.receive(pacote);
            String msg = new String(pacote.getData(), 0, pacote.getLength(), StandardCharsets.UTF_8);
            return new Resultado.Sucesso(new PacoteRecebido(msg, pacote.getAddress(), pacote.getPort()));
        } catch (IOException e) {
            return new Resultado.Falha(e.getMessage());
        }
    }

    private static void enviarResposta(DatagramSocket socket,
                                       InetAddress destino, int portaDestino) throws IOException {
        byte[] bytes = RESPOSTA.getBytes(StandardCharsets.UTF_8);
        socket.send(new DatagramPacket(bytes, bytes.length, destino, portaDestino));
    }

    public static void iniciarServidor() {
        System.out.printf("""
            Servidor UDP iniciado
            Escutando na porta %d com buffer de %d bytes...
            Pressione CTRL+C para encerrar.
            """, PORTA, BUFFER_SIZE);

        try (var socket = new DatagramSocket(PORTA)) {
            byte[] buffer = new byte[BUFFER_SIZE];

            while (!Thread.currentThread().isInterrupted()) {
                System.out.println("Aguardando pacote...");

                switch (receberPacote(socket, buffer)) {
                    case Resultado.Sucesso(var p) -> {
                        System.out.printf("Mensagem de %s:%d -> %s\n",
                            p.origem().getHostAddress(), p.portaOrigem(), p.mensagem());
                        enviarResposta(socket, p.origem(), p.portaOrigem());
                    }
                    case Resultado.Falha(var motivo) ->
                        System.err.println("Falha ao receber pacote: " + motivo);
                }
            }
        }
    }
}
```

```

    } catch (IOException e) {
        System.err.println("Erro no servidor: " + e.getMessage());
    }
}

public static void main(String[] args) {
    // Criando uma thread virtual para o servidor UDP
    try {
        Thread.ofVirtual()
            .name("servidor-udp")
            .start(ServidorUdp::iniciarServidor)
            .join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    } // Aguarda a thread do servidor terminar (neste caso, ela roda indefinidamente)
}
}

```

Listagem 7: Cliente UDP

```

package engtelecom.std.udp;

import module java.base; // Java 25: importação de módulo

public class ClienteUdp {

    private static final int BUFFER_SIZE = 1024;

    public sealed interface Resultado permits Resultado.Sucesso, Resultado.Falha {
        record Sucesso(String resposta) implements Resultado {
        }
        record Falha(String motivo) implements Resultado {
        }
    }

    public static Resultado comunicar(String[] config, String mensagem) {

        byte[] bufferEnvio = mensagem.getBytes(StandardCharsets.UTF_8);
        byte[] bufferRecepcao = new byte[BUFFER_SIZE];

        try (var socket = new DatagramSocket()) {
            var enderecoServidor = InetAddress.getByName(config[0]);
            int portaServidor = Integer.parseInt(config[1]);

            // Envio
            var pacoteEnvio = new DatagramPacket(
                bufferEnvio, bufferEnvio.length, enderecoServidor, portaServidor);
            socket.send(pacoteEnvio);
            System.out.println("Mensagem enviada: " + mensagem);

            // Recepção
            System.out.println("Aguardando resposta...");
            var pacoteResposta = new DatagramPacket(bufferRecepcao, bufferRecepcao.length);
            socket.receive(pacoteResposta);

            String resposta = new String(
                pacoteResposta.getData(), 0, pacoteResposta.getLength(), StandardCharsets.UTF_8);

            return new Resultado.Sucesso(resposta);
        } catch (IOException e) {

```



```

        return new Resultado.Falha(e.getMessage());
    }
}

public static void main(String[] args) {
    System.out.println("Cliente UDP iniciado...");

    var config = switch (args.length) {
        case 2 -> new String[]{ args[0], args[1] };
        default -> new String[]{ "localhost", "9876" };
    };

    switch (comunicar(config, "Olá, eu sou o cliente UDP!")) {
        case Resultado.Sucesso(var resposta) ->
            System.out.println("Resposta do servidor: " + resposta);
        case Resultado.Falha(var motivo) ->
            System.err.println("Falha na comunicação: " + motivo);
    }
}
}

```

3 Multicast

Acesse <https://github.com/emersonmello/sockets-java> e execute o exemplo de *multicast* com o servidor e cliente em contêineres Docker diferentes. Mas atenção, é necessário que ambos estejam na mesma rede. Não há ordem específica para execução do servidor e cliente.